

ADR-002 - Confirmacao resiliente, notificacao assincrona e escala

Status: Accepted | Projeto: Atlas Ticketing | Escopo: Fases 3-5

Contexto

A confirmacao de uma compra precisa produzir um unico resultado logico (Order + Tickets), enquanto o provider HTTP de notificacao e propositalmente instavel: pode ficar lento, responder 5xx, ficar indisponivel ou entregar duplicado. A falha externa nao pode manter transacoes abertas, perder silenciosamente a notificacao nem bloquear novas compras. O desenho tambem precisa funcionar com multiplas instancias da API e ter uma evolucao clara para 10x/100x.

Decisao

- **Transacao curta no MySQL.** A confirmacao trava Reservation e Seats, revalida ownership/TTL e grava Order, Ticket(s), OutboxEvent e NotificationDelivery no mesmo commit. orders.reservation_id e unico; repeticoes retornam a mesma compra.
- **Sem HTTP no commit.** O endpoint devolve 201/200 depois do commit. Um relay agenda Outbox pendente para Redis e o Horizon executa a chamada HTTP fora da request.
- **At-least-once + idempotencia.** O job usa um notification_id estavel, enviado como Idempotency-Key. O provider deduplica retries ja aceitos. Nao e prometido exactly-once.
- **Falha observavel.** Connect timeout 1s, timeout 2s, 5 tentativas e backoff 5/15/30/60s. Apos esgotar retries, Delivery vira failed e pode ser reprocessada sem recriar Order/Ticket.
- **LGPD.** CPF fica cifrado no Ticket e nao entra em Outbox, logs, audit trail ou payload do provider. Codigo do ingresso e opaco (UUID).

Fluxo

Request sincrona	Processamento assincrono
Nginx -> PHP-FPM -> lock MySQL	Scheduler -> Outbox -> Redis
Order + Tickets + Outbox + Delivery	Horizon -> SendTicketNotification
COMMIT -> HTTP 201/200	HTTP Provider -> sent / retry / failed

A janela "provider aceitou, processo caiu antes de persistir sent" continua possivel. Por isso retries precisam ser idempotentes no destino. Esta e a consequencia assumida da semantica at-least-once.

Alternativas consideradas

- **HTTP sincrono na confirmacao:** rejeitado porque acopla latencia/disponibilidade externa ao checkout e alonga a transacao.
- **Job depois do commit sem Outbox:** rejeitado porque existe uma janela entre commit e enqueue onde a notificacao pode desaparecer silenciosamente.
- **Exactly-once:** rejeitado como promessa. MySQL, Redis e HTTP nao fornecem transacao distribuida end-to-end; idempotencia e a garantia realista.
- **Kafka agora:** rejeitado por custo operacional sem necessidade de replay/multiplos consumidores. Redis/Horizon cobre o escopo; SQS/Kafka entram quando a semantica justificar.

Consequencias operacionais

- Provider lento/down nao reverte compra e nao bloqueia novos checkouts.
- Operacao precisa acompanhar Horizon, Pulse, notification_deliveries e /api/ops/metrics.
- A recuperacao de falha prolongada e explicita: notifications:retry-failed, preservando o historico de tentativas.
- Auditoria append-only registra mudancas de negocio sem CPF; alertas de failure rate sao gerados automaticamente pelo Scheduler.

Multiplas instancias e 10x

A API e stateless. Varias replicas podem ficar atras de um load balancer porque ownership e concorrência convergem no MySQL compartilhado, nao na memoria da instancia. Horizon pode escalar por filas e Redis/MySQL podem ser servicos gerenciados. Relatorios e outras leituras podem usar read replicas ou read models; a escrita critica continua no primario.

100x

- **Entrada:** waiting room/virtual queue e rate limiting por comprador/evento para reduzir thundering herd.
- **Leitura:** CDN/cache para catalogo; read models agregados para relatorios; busca secundaria (OpenSearch/Elasticsearch) reconstruível.
- **Escrita:** isolamento de pools por evento quente e, quando necessario, particionamento/sharding por event_id. O banco continua sendo a autoridade final de unicidade.
- **Mensageria:** filas gerenciadas (ex.: SQS) para jobs; Kafka apenas quando replay, log duravel e multiplos consumidores justificarem.
- **Observabilidade:** OpenTelemetry como camada portavel para exportar traces/metrics a Datadog/Grafana sem acoplar o dominio.

Melhoria de regra de produto

Fila virtual com limite de assentos por comprador e uma janela curta de checkout. Isso reduz disputa, abuso e a experiencia de "clique perdido" sem enfraquecer a garantia final de unicidade no MySQL. A fila controla admissao; o lock/constraint continua protegendo a verdade.

Verificacao

Risco	Evidencia/control
Double booking	FOR UPDATE + transacao + k6 50 requests / 10 seats + validacao SQL
Ticket duplicado	UNIQUE reservation_id em orders + confirmacao idempotente
Notificacao perdida	Outbox e Delivery no mesmo commit da compra
Provider indisponivel	Timeout/retry/backoff/failed + reprocessamento explicito
Vazamento de CPF	Cast encrypted; nao enviado a Outbox/provider/audit

Decisao final: manter o checkout sincronamente dependente apenas das invariantes locais do MySQL; tratar efeitos externos como assincronos, observaveis, retryable e idempotentes.